

Centro de Procesamiento de Datos



UNIVERSIDAD
DE GRANADA

Práctica 4. Docker Swarm

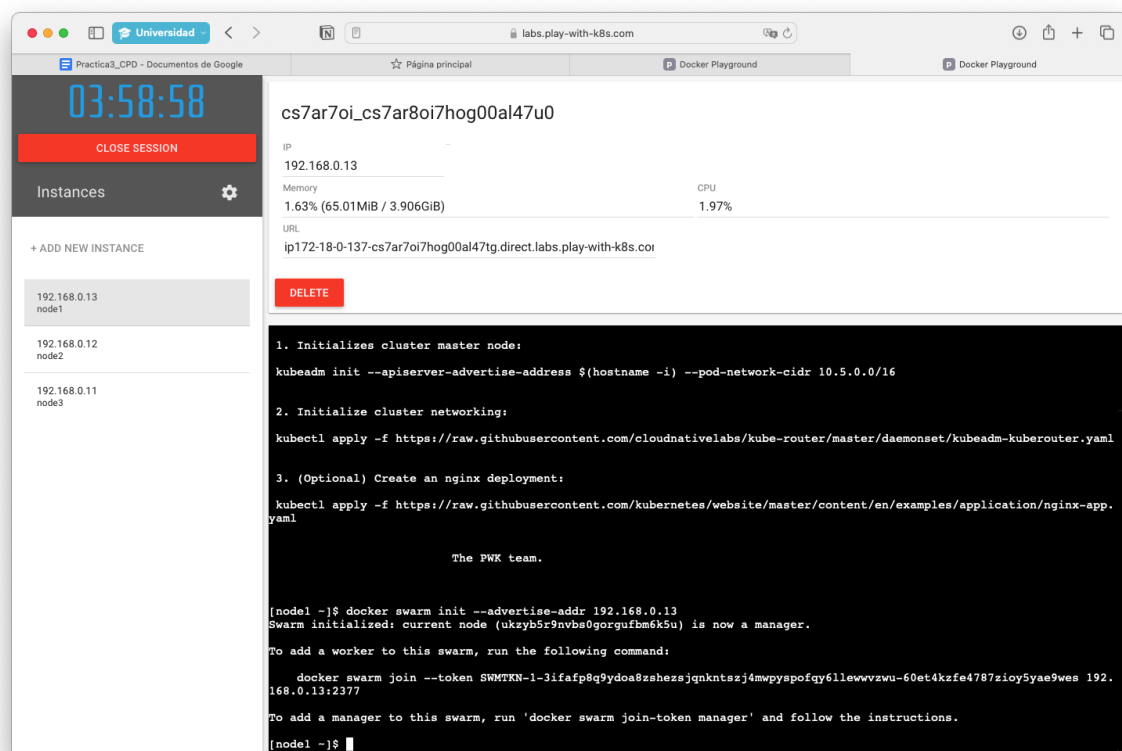
Juan Navarro Maldonado

Índice:

Creación de las máquinas virtuales con el acceso al laboratorio remoto e inicio del manager de Docker Swarm.....	3
Ejecución del servicio web.....	5
3 Nodos activos.....	6
Cambio de escala a 2.....	7
Apagamos nodo activo y ejecución de un nodo.....	8
Activación automática del segundo nodo.....	9

Creación de las máquinas virtuales con el acceso al laboratorio remoto e inicio del manager de Docker Swarm

Accedemos al laboratorio virtual y haciendo click en el apartado ADD NEW INSTANCE, esta se denominará como 'node 1'. Con este inicializamos nuestro **swarm** utilizando el comando `docker swarm init`. En nuestro caso, el nodo 'node1' actúa como el **nodo principal o manager** del swarm, es decir, el encargado de gestionar el clúster y coordinar las tareas distribuidas entre otros nodos que se añadan como trabajadores. La opción `--advertise-addr` se utiliza para especificar la dirección IP que otros nodos utilizarán para conectarse al nodo principal, permitiendo así la creación de un entorno de balanceo de carga distribuido.



Para añadir redundancia al clúster Docker Swarm, creamos otros dos nodos llamados **node2** y **node3**. Después de haber inicializado el swarm en **node1**, estos nuevos nodos se pueden unir como trabajadores utilizando el comando `docker swarm join --token <TOKEN> <IP_MANAGER>:<PORT>`, donde **<TOKEN>** es el token generado por **node1** y **<IP_MANAGER>:<PORT>** es la dirección IP y el puerto del nodo principal. Al ejecutar este comando en **node2** y **node3**, ambos nodos se conectarán al clúster y recibirán tareas distribuidas por el nodo principal, mejorando la disponibilidad y escalabilidad del sistema.

03:58:24

CLOSE SESSION

Instances

+ ADD NEW INSTANCE

192.168.0.13
node1

192.168.0.12
node2

192.168.0.11
node3

cs7ar7oi_cs7arg8i7hog00al47ug

IP
192.168.0.12

Memory
1.51% (60.24MiB / 3.906GiB)

CPU
0.75%

URL
ip172-18-0-139-cs7ar7oi7hog00al47tg.direct.labs.play-with-k8s.com

DELETE

WARNING!!!!

This is a sandbox environment. Using personal credentials is HIGHLY discouraged. Any consequences of doing so, are completely the user's responsibilities.

You can bootstrap a cluster as follows:

1. Initialize cluster master node:
`kubeadm init --apiserver-advertise-address $(hostname -i) --pod-network-cidr 10.5.0.0/16`
2. Initialize cluster networking:
`kubectctl apply -f https://raw.githubusercontent.com/cloudnativelabs/kube-router/master/daemonset/kubeadm-kuberouter.yaml`
3. (Optional) Create an nginx deployment:
`kubectctl apply -f https://raw.githubusercontent.com/kubernetes/website/master/content/en/examples/application/nginx-app.yaml`

The PWK team.

```
[node2 ~]$ docker swarm join --token SWMTKN-1-3ifafp8q9ydoa8szshezsjqnkntszj4mwpypofqy6llewwvzwu-60et4kzfe4787zioy5yae9w
es 192.168.0.13:2377
This node joined a swarm as a worker.
[node2 ~]$
```

Por último, podemos utilizar el comando `docker node ls` para verificar que los nodos se han añadido correctamente al clúster y están funcionando como se espera.

03:57:47

CLOSE SESSION

Instances

+ ADD NEW INSTANCE

192.168.0.13
node1

192.168.0.12
node2

192.168.0.11
node3

cs7ar7oi_cs7ar8oi7hog00al47u0

IP
192.168.0.13

Memory
1.79% (71.65MiB / 3.906GiB)

CPU
9.71%

URL
ip172-18-0-137-cs7ar7oi7hog00al47tg.direct.labs.play-with-k8s.com

DELETE

2. Initialize cluster networking:
`kubectctl apply -f https://raw.githubusercontent.com/cloudnativelabs/kube-router/master/daemonset/kubeadm-kuberouter.yaml`

3. (Optional) Create an nginx deployment:
`kubectctl apply -f https://raw.githubusercontent.com/kubernetes/website/master/content/en/examples/application/nginx-app.yaml`

The PWK team.

```
[node1 ~]$ docker swarm init --advertise-addr 192.168.0.13
Swarm initialized: current node (ukzyb5r9nvbs0gorgufbm6k5u) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-3ifafp8q9ydoa8szshezsjqnkntszj4mwpypofqy6llewwvzwu-60et4kzfe4787zioy5yae9wes 192.168.0.13:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

[node1 ~]$ docker node ls
ID                                HOSTNAME    STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
ukzyb5r9nvbs0gorgufbm6k5u *      node1      Ready    Active           Leader             24.0.2
pbi4u0v5exxe3njyiprb7hvs9       node2      Ready    Active           Leader             24.0.2
dquogi8i6rrhflt03o3w4hvv        node3      Ready    Active           Leader             24.0.2
[node1 ~]$
```

Ejecución del servicio web

El comando `docker service create --name web --replicas 3 --mount type=bind,src=/etc/hostname,dst=/usr/share/nginx/html/index.html,readonly --publish published=8080,target=80 nginx` crea un servicio en Docker Swarm llamado `web` con 3 réplicas del servidor NGINX. Este servicio monta el archivo del sistema host `/etc/hostname` dentro de cada contenedor en la ruta `/usr/share/nginx/html/index.html`, lo que significa que el contenido del archivo del `hostname` se mostrará en la página principal del servidor web. Además, el servicio expone el puerto 8080 del host y lo redirige al puerto 80 dentro del contenedor, lo que permite acceder al servicio NGINX desde el navegador a través del puerto 8080. El uso de `--replicas 3` asegura que se ejecuten tres instancias del servicio, proporcionando alta disponibilidad.

Al ejecutar el comando `docker ps` tras haber creado el servicio web con `docker service create`, no veremos directamente el servicio `web` listado como tal. Para visualizar los contenedores individuales que forman parte de este servicio, el comando adecuado sería `docker ps`. Esto mostrará los contenedores en ejecución, incluyendo las 3 réplicas del servicio `web`. En la lista de contenedores, se puede identificar cada réplica del servicio `web` con un nombre similar a `web.1`, `web.2`, y `web.3`, junto con detalles como el ID del contenedor, la imagen utilizada (NGINX), los puertos publicados, y el estado de cada uno.

The screenshot shows the Docker Playground interface. On the left, there's a sidebar with a clock showing 03:55:20, a 'CLOSE SESSION' button, and a list of instances. The main area displays the details of a Docker Swarm cluster named 'cs7ar7oi_cs7ar8oi7hog00al47u0'. It shows the IP address 192.168.0.13, a published port 8080, and the URL 'ip172-18-0-137-cs7ar7oi7hog00al47tg.direct.labs.play-with-k8s.com'. Below this, there's a 'DELETE' button and a terminal window showing the commands and output for creating and managing the swarm and the 'web' service.

```
To add a worker to this swarm, run the following command:
docker swarm join --token SWMTKN-1-3ifafp8g9ydoa8zshesjgnkntszj4mwpyspofgy61lewwvzwu-60et4kzfe4787ziocy5yae9wes 192.168.0.13:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

[node1 ~]$ docker node ls
ID                                HOSTNAME    STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
ukzyb5r9nvbs0gorgufbm6k5u *     node1      Ready    Active          Leader            24.0.2
pbi4u0v5exxe3njyiprb7hvs9       node2      Ready    Active          Leader            24.0.2
dguogi8i6rrhlt03o3v4hvv         node3      Ready    Active          Leader            24.0.2
[node1 ~]$ docker service create --name web --replicas 3 --mount type=bind,src=/etc/hostname,dst=/usr/share/nginx/html/index.html,readonly --publish published=8080,target=80 nginx
zc9y3x83fslwbbxw3o1s6s3fx7
overall progress: 3 out of 3 tasks
1/3: running [=====]
2/3: running [=====]
3/3: running [=====]
verify: Service converged
[node1 ~]$ curl http://192.168.0.13:8080
node1
[node1 ~]$ curl http://192.168.0.13:8080
node3
[node1 ~]$ curl http://192.168.0.13:8080
node2
[node1 ~]$ docker service ps web
ID                                NAME        IMAGE        NODE    DESIRED STATE    CURRENT STATE    ERROR    PORTS
qvjzwekt4u0v web.1       nginx:latest node3    Running          Running about a minute ago
tga6mknjogdb web.2       nginx:latest node1    Running          Running about a minute ago
e30fwdktjxhp web.3       nginx:latest node2    Running          Running about a minute ago
[node1 ~]$
```

3 Nodos activos

Para verificar que las tres réplicas del servicio web están activas y funcionando correctamente, puedes utilizar el comando `curl http://localhost:8080`. Este comando realiza una solicitud HTTP al servicio NGINX que se ejecuta en el puerto 8080 del host. Si las tres réplicas están operativas, deberías recibir una respuesta que incluya el contenido del archivo montado, que en este caso sería el contenido de `/etc/hostname` del sistema host. Si el comando se ejecuta sin errores y devuelve el contenido esperado, significa que las réplicas del servicio están activas y accesibles a través del puerto especificado.

The screenshot shows the Docker Playground interface in a web browser. The top bar includes a clock showing 03:56:19, a 'CLOSE SESSION' button, and a list of instances. The main panel displays the details for a specific instance, 'cs7ar7oi_cs7ar8oi7hog00al47u0', with IP 192.168.0.13 and port 8080. Below this, a terminal window shows the commands and output for initializing a Docker Swarm cluster with 3 nodes. The output indicates that the cluster is initialized, and the service 'web' is running on all 3 nodes. The terminal output is as follows:

```
[node1 -]$ docker swarm init --advertise-addr 192.168.0.13
Swarm initialized: current node (ukzyb5r9nvbs0gorgufbm6k5u) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-3ifafp8q9ydoa8szheszjqnktsszj4mwpyspofgy61lewwvzwu-60et4kzfe4787ziocy5yae9wes 192.168.0.13:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

[node1 -]$ docker node ls
ID                                HOSTNAME    STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
ukzyb5r9nvbs0gorgufbm6k5u *     node1      Ready    Active           Leader             24.0.2
pb14u0v5exxe3njyiprb7hvs9      node2      Ready    Active           -                  24.0.2
dquogi8i6rrhflt03o3w4hvv       node3      Ready    Active           -                  24.0.2
[node1 -]$ docker service create --name web --replicas 3 --mount type=bind,src=/etc/hostname,dst=/usr/share/nginx/html/index.html,readonly --publish published=8080,target=80 nginx
zc9yx83fslwbbxw3o1s6s3fx7
overall progress: 3 out of 3 tasks
1/3: running [=====]
2/3: running [=====]
3/3: running [=====]
verify: Service converged
[node1 -]$ curl http://192.168.0.13:8080
node1
[node1 -]$ curl http://192.168.0.13:8080
node3
[node1 -]$ curl http://192.168.0.13:8080
node2
[node1 -]$
```

Cambio de escala a 2

El comando `docker service scale web=2` se utiliza para ajustar la escala del servicio `web` a 2 réplicas. Al ejecutar este comando, Docker reducirá la cantidad de instancias del servicio `web` a 2, deteniendo cualquier réplica adicional que estuviera en ejecución. Este ajuste permite gestionar de manera dinámica la capacidad del servicio según la demanda, lo que es útil para optimizar el uso de recursos en el clúster. Al verificar nuevamente con `docker service ls` o `docker ps`, podrás ver que el servicio ahora tiene solo 2 réplicas activas.

03:49:54

CLOSE SESSION

Instances

+ ADD NEW INSTANCE

192.168.0.13
node1

192.168.0.12
node2

192.168.0.11
node3

cs7ar7oi_cs7ar8oi7hog00al47u0

IP
192.168.0.13

Memory
3.14% (125.6MiB / 3.906GiB)

CPU
1.75%

URL
ip172-18-0-137-cs7ar7oi7hog00al47tg.direct.labs.play-with-k8s.com

DELETE

```
index.html,readonly --publish published=8080,target=80 nginx
zcY3x83fslwbbxw3ois6s3fx7
overall progress: 3 out of 3 tasks
1/3: running [=====>]
2/3: running [=====>]
3/3: running [=====>]
verify: Service converged
[node1 -] $ curl http://192.168.0.13:8080
node1
[node1 -] $ curl http://192.168.0.13:8080
node3
[node1 -] $ curl http://192.168.0.13:8080
node2
[node1 -] $ docker service ps web
ID            NAME      IMAGE      NODE    DESIRED STATE  CURRENT STATE      ERROR      PORTS
qvj3wekt4u0v  web.1     nginx:late node3     Running        Running about a minute ago
qga6mknjogdb  web.2     nginx:late node1     Running        Running about a minute ago
e30fwdktjxbp  web.3     nginx:late node2     Running        Running about a minute ago
sdasdas
bash: sdasdas: command not found
[node1 -] $ docker service ps web
ID            NAME      IMAGE      NODE    DESIRED STATE  CURRENT STATE      ERROR      PORTS
qvj3wekt4u0v  web.1     nginx:late node3     Running        Running 6 minutes ago
qga6mknjogdb  web.2     nginx:late node1     Running        Running 6 minutes ago
e30fwdktjxbp  web.3     nginx:late node2     Running        Running 6 minutes ago
[node1 -] $ docker service scale web=2
web scaled to 2
overall progress: 2 out of 2 tasks
1/2: running [=====>]
2/2: running [=====>]
verify: Service converged
[node1 -] $
```

The screenshot shows the Docker Playground interface. On the left, there's a sidebar with a clock showing 03:49:23, a 'CLOSE SESSION' button, and a list of instances: 192.168.0.13 (node1), 192.168.0.12 (node2), and 192.168.0.11 (node3). The main panel shows the session ID 'cs7ar7oi_cs7ar8oi7hog00al47u0' and its details: IP 192.168.0.13, Memory 3.14% (125.7MiB / 3.906GiB), CPU 1.06%, and a URL. Below this is a 'DELETE' button. The terminal output shows the following commands and results:

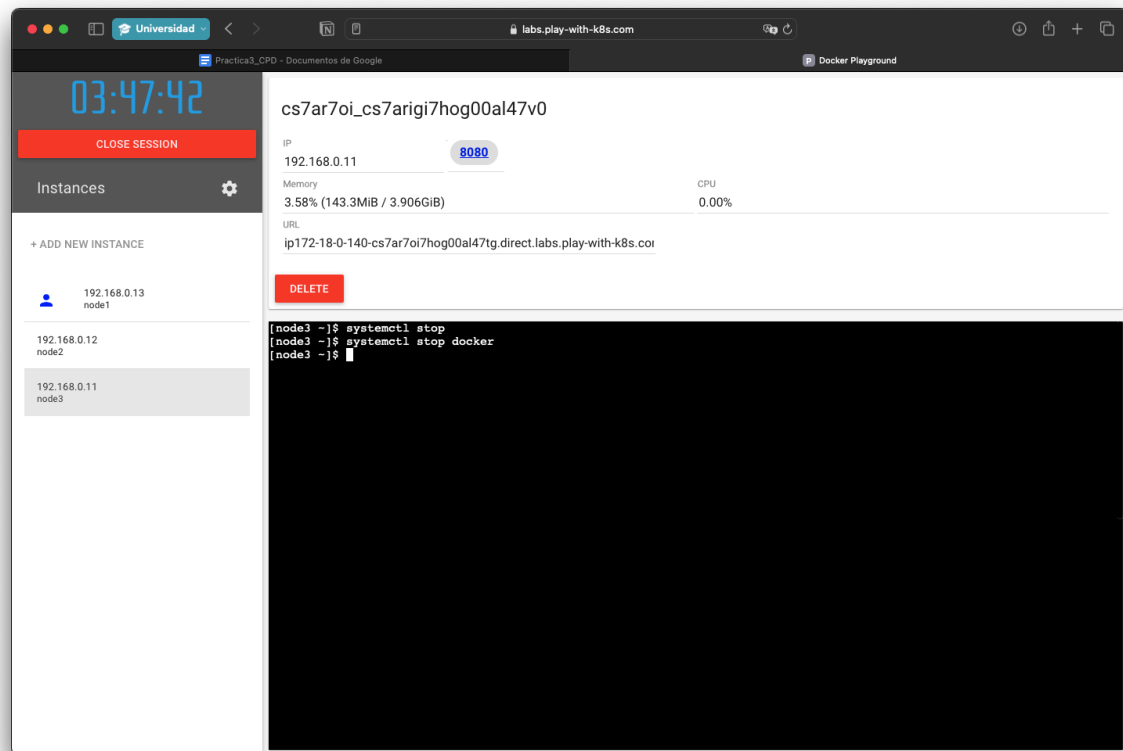
```

bash: sdaadas: command not found
[node1 -] $ docker service ps web
ID            NAME      IMAGE      NODE    DESIRED STATE  CURRENT STATE      ERROR      PORTS
qvjzwekt4u0v  web.1     nginx:late node3     Running        Running about a minute ago
gga6mknjogdb  web.2     nginx:late node1     Running        Running about a minute ago
e30fwdktjxbp  web.3     nginx:late node2     Running        Running about a minute ago
sdaadas
[node1 -] $ docker service scale web=2
web scaled to 2
overall progress: 2 out of 2 tasks
1/2: running [=====]
2/2: running [=====]
verify: Service converged
[node1 -] $ curl http://192.168.0.13:8080
node1
[node1 -] $ curl http://192.168.0.13:8080
node3
[node1 -] $ curl http://192.168.0.13:8080
node1
[node1 -] $ curl http://192.168.0.13:8080
node1
[node1 -] $ curl http://192.168.0.13:8080
node3
[node1 -] $ curl http://192.168.0.13:8080
node1
[node1 -] $

```

Apagamos nodo activo y ejecución de un nodo

Al ejecutar el comando `systemctl stop docker` en el nodo 2, detendremos el servicio Docker en ese nodo, lo que hará que se detengan todos los contenedores y servicios en ejecución en él, incluida la réplica del servicio `web`. Sin embargo, debido a la configuración de Docker Swarm, el nodo 3 debería encargarse automáticamente de las tareas que estaban asignadas al nodo 2. Esto significa que, al verificar el estado del servicio con `docker service ls`, podrás observar que el número de réplicas activas sigue siendo 2, ya que el Swarm redistribuirá la carga y levantará una nueva réplica del servicio `web` en el nodo 3 para mantener la disponibilidad del servicio. Esto demuestra la capacidad de Docker Swarm para manejar la alta disponibilidad y la tolerancia a fallos dentro del clúster.



Activación automática del segundo nodo

Después de detener el nodo 2 con `systemctl stop docker`, puedes verificar la disponibilidad del servicio utilizando `curl http://localhost:8080` desde el nodo 3. Al ejecutar este comando, deberías recibir una respuesta del servicio `web`, indicando que la réplica en el nodo 3 está activa y funcionando. Esto confirmará que, a pesar de la detención del nodo 2, Docker Swarm ha redistribuido las réplicas y que el servicio sigue accesible en el puerto 8080. Si el comando `curl` devuelve el contenido esperado (el hostname del sistema o el contenido montado), significa que el clúster está manejando correctamente la alta disponibilidad y la recuperación ante fallos.

03:47:32

CLOSE SESSION

Instances

+ ADD NEW INSTANCE

192.168.0.13
node1

192.168.0.12
node2

192.168.0.11
node3

cs7ar7oi_cs7ar8oi7hog00al47u0

IP

192.168.0.13

8080

Memory

3.14% (125.8MiB / 3.906GiB)

CPU

1.02%

URL

ip172-18-0-137-cs7ar7oi7hog00al47tg.direct.labs.play-with-k8s.co

DELETE

```
node3
[node1 ~]$ curl http://192.168.0.13:8080
node1
[node1 ~]$ curl http://192.168.0.13:8080
node3
[node1 ~]$ curl http://192.168.0.13:8080
node1
[node1 ~]$ curl http://192.168.0.13:8080
node3
[node1 ~]$ curl http://192.168.0.13:8080
node1
[node1 ~]$ docker-machine stop node3
bash: docker-machine: command not found
[node1 ~]$ docker-machine stop node3
bash: docker-machine: command not found
[node1 ~]$ curl http://192.168.0.13:8080
node1
[node1 ~]$ curl http://192.168.0.13:8080
node1
[node1 ~]$ curl http://192.168.0.13:8080
node1
[node1 ~]$ curl http://192.168.0.13:8080
node1
[node1 ~]$ curl http://192.168.0.13:8080
node1
[node1 ~]$ curl http://192.168.0.13:8080
node1
[node1 ~]$ curl http://192.168.0.13:8080
node2
[node1 ~]$ curl http://192.168.0.13:8080
node1
[node1 ~]$ curl http://192.168.0.13:8080
node2
[node1 ~]$
```